

AEGIS — Setup Checklist

Personal environment setup tasks for building the MVP/POC. Complete these in order — each section unblocks the next.

1. Local Development Environment

- ☐ Install Go 1.23+ (`brew install go`)
- ☐ Install Terraform 1.5+ (`brew install terraform`)
- ☐ Verify Node.js 20+ and npm are installed (`node --version`)
- ☐ Install Docker Desktop (for building/testing containers locally)
- ☐ Clone the AEGIS repo and verify the monorepo structure

2. GCP Project Setup

- ☐ Create a new GCP project (e.g., `aegis-dev`) in your personal GCP account
- ☐ Link a billing account to the project (free tier covers most dev usage)
- ☐ Install the gcloud CLI (`brew install google-cloud-sdk`)
- ☐ Authenticate: `gcloud auth login` and `gcloud auth application-default login`
- ☐ Set default project: `gcloud config set project aegis-dev`

3. Terraform — Project Bootstrap

- ☐ Copy `terraform/project/terraform.tfvars.example` to `terraform/project/terraform.tfvars`
- ☐ Fill in your `project_id` , `region` , and `billing_account`
- ☐ Run `terraform init` in `terraform/project/`
- ☐ Run `terraform plan` and review the output
- ☐ Run `terraform apply` to enable all required GCP APIs
- ☐ Verify APIs are enabled: `gcloud services list --enabled`

4. Terraform — Infrastructure

- ☐ Copy `terraform/infra/terraform.tfvars.example` to `terraform/infra/terraform.tfvars`
- ☐ Fill in your `project_id` and `region`
- ☐ Run `terraform init` in `terraform/infra/`
- ☐ Run `terraform plan` and review
- ☐ Run `terraform apply` to create Healthcare API DICOM stores, GCS bucket, Pub/Sub
- ☐ Verify DICOM store exists: `gcloud healthcare dicom-stores list --dataset=aegis --location=us-central1`
- ☐ Verify staging bucket exists: `gsutil ls`

5. Sample DICOM Data for Local Testing

- ☐ Download sample brain MRI DICOM files for testing (options below):
 - TCIA (The Cancer Imaging Archive): <https://www.cancerimagingarchive.net/>
 - dicom.offis.de sample files: <https://dicom.offis.de/dcmtdk/>
 - NEMA WG-6 sample DICOM: <ftp://medical.nema.org/medical/dicom/DataSets/>
- ☐ Place sample files in a local test directory (not committed to git)

- ☐ Verify files open in a DICOM viewer (e.g., Horos, 3D Slicer) to confirm validity

6. Frontend — Upload Portal

- ☐ `cd frontend/upload-portal && npm install`
- ☐ `npm run dev` — verify it runs on <http://localhost:3000>
- ☐ Install DICOM libraries: `npm install dcmjs dicom-parser`
- ☐ Test parsing a sample DICOM file in the browser console

7. Frontend — Admin Dashboard

- ☐ `cd frontend/admin-dashboard && npm install`
- ☐ `npm run dev` — verify it runs on <http://localhost:3001>
- ☐ (Later) Add OHIF Viewer dependency once DICOM stores are populated

8. Go API

- ☐ `cd api && go run .` — verify health endpoint at <http://localhost:8080/healthz>
- ☐ Add GCP SDK dependencies: `go get cloud.google.com/go/storage cloud.google.com/go/healthcare`
- ☐ Implement signed URL generation endpoint
- ☐ Test upload flow: browser → signed URL → GCS staging bucket

9. GitHub Repository

- ☐ Verify remote is set: `git remote -v`
- ☐ Push monorepo scaffold to `develop` branch
- ☐ Set up branch protection on `main` (require PR reviews)
- ☐ (Optional) Set up GitHub Actions for CI (lint, typecheck, Go test)

10. Future — Before Proposing to Work

- ☐ Have a working end-to-end demo: upload → anonymize → view in OHIF
- ☐ Prepare a 5-minute screen recording of the demo flow
- ☐ Draft a one-page proposal covering: problem, solution, differentiation, cost estimate
- ☐ Identify potential pilot users / departments at your institution
- ☐ Research your company's internal startup / innovation program requirements

Generated 2026-02-17. See *ARCHITECTURE.md* for the full system design.